Luboss95 Village Banking v2 (LUBOSS 95)

A comprehensive FastAPI-based Village Banking system with PostgreSQL, double-entry accounting, RBAC, and AI-powered chat assistance.

Overview

LUBOSS 95 is a modern microfinance management system designed for village banking operations. It features double-entry accounting, role-based access control, loan management, member declarations, deposit tracking, and an AI assistant constrained to constitution/policies and member account information.

Features

Core Functionality

- **Double-entry accounting ledger** for financial accuracy and audit trails
- **Role-Based Access Control (RBAC)** with 6 roles: Admin, Chairman, Vice-Chairman, Treasurer, Compliance, Member
- **RAG-based AI chat** constrained to constitution/rules/policies and member's own account status
- **Loan Management** - Application, approval, disbursement, repayment tracking, and history
- **Member Declarations** - Monthly declarations with deposit proof uploads
- **Cycle Management** - Annual financial cycles with configurable phases
- **Credit Rating System** - Tiered credit ratings with borrowing limits and interest rates
- **Deposit Proof Workflow** - Upload, review, approve/reject with comments
- **Penalty Management** - Compliance tracking and treasurer approval
- **Seamless Migration** - Tools to migrate from legacy MySQL system

User Interfaces

- **Next.js Frontend** - Modern, mobile-first UI with light blue theme
- **Member Dashboard** - Account status, declarations, loans, deposits
- **Treasurer Dashboard** - Deposit approvals, loan management, active loan monitoring
- **Chairman Dashboard** - Member management, cycle management, user roles
- **Admin Dashboard** - System settings and configuration

Tech Stack

Backend

- **Python 3.11+** with FastAPI
- **PostgreSQL 17** with pgvector extension for vector search
- **SQLAlchemy 2.0** (ORM)
- **Alembic** (Database migrations)
- **Pydantic v2** (Data validation)
- **Uvicorn** (ASGI server)

Frontend

- **Next.js** (React framework)
- **TypeScript**
- **Tailwind CSS** (Styling)
- **React Context API** (State management)

AI & Search

- **Groq API** with Llama 3.1 70B (LLM)
- **OpenAI** text-embedding-3-small (Embeddings)
- **pgvector** (Vector similarity search)
- **RAG Architecture** (Retrieval Augmented Generation)

Project Structure

```
luboss-vb/
├── app/                    # Backend application
│   ├── api/                # FastAPI routers
│   │   ├── admin.py        # Admin endpoints
│   │   ├── auth.py         # Authentication
│   │   ├── chairman.py     # Chairman endpoints
│   │   ├── compliance.py   # Compliance endpoints
│   │   ├── member.py       # Member endpoints
│   │   ├── treasurer.py    # Treasurer endpoints
│   │   └── ai.py           # AI chat endpoints
│   ├── core/               # Core configuration
│   │   ├── config.py       # Environment config
│   │   ├── dependencies.py # FastAPI dependencies
│   │   └── security.py     # JWT & password hashing
│   ├── db/                 # Database connection
│   ├── models/             # SQLAlchemy ORM models
│   ├── schemas/            # Pydantic schemas
│   ├── services/           # Business logic
│   │   ├── accounting.py   # Ledger operations
│   │   ├── auth.py         # Authentication logic
│   │   ├── cycle.py        # Cycle management
│   │   ├── member.py       # Member operations
│   │   ├── policy.py       # Policy calculations
│   │   ├── rbac.py         # Role-based access
│   │   └── transaction.py  # Transaction processing
│   ├── ai/                 # AI/RAG functionality
│   │   ├── chat.py         # Chat endpoint
│   │   ├── ingestion.py    # Document ingestion
│   │   ├── retrieval.py   # Vector search
│   │   └── tools.py        # AI tool contracts
│   ├── main.py             # FastAPI app entry point
│   └── requirements.txt    # Python dependencies
│
├── ui/                     # Frontend application
│   ├── app/                # Next.js app directory
│   │   ├── dashboard/      # Dashboard pages
│   │   │   ├── admin/      # Admin dashboard
```

```
│   │   │   ├── chairman/   # Chairman dashboard
│   │   │   ├── member/     # Member dashboard
│   │   │   └── treasurer/  # Treasurer dashboard
│   │   ├── login/          # Login page
│   │   ├── register/       # Registration page
│   │   └── pending/        # Pending approval page
│   ├── contexts/           # React contexts
│   ├── lib/                 # Utilities
│   └── package.json        # Node dependencies
│
├── alembic/                # Database migrations
│   └── versions/           # Migration scripts
│
├── docs/                   # Documentation
│   ├── ai/                 # AI/RAG documentation
│   ├── migration/          # Migration guides
│   ├── accounting_and_rules.md  # Chart of accounts and system rules
│   └── source/             # Source documents (constitution, policies)
│
├── scripts/                # Utility scripts
│   ├── create_admin.py     # Create admin user
│   ├── seed_data.py        # Seed initial data
│   └── setup_ledger_accounts.py  # Setup ledger accounts
│
└── uploads/                # Uploaded files
    ├── constitution/       # Constitution PDFs
    └── deposit_proofs/     # Deposit proof documents
```

Setup Instructions

Prerequisites

- Python 3.11 or higher
- PostgreSQL 17 or higher
- Node.js 18+ and npm
- Git

Backend Setup

1. **Clone the repository**:

```
git clone <repository-url>
cd luboss-vb
```

2. **Create and activate virtual environment**:

```
cd app
python3 -m venv venv
source venv/bin/activate  # On Windows: venv\Scripts\activate
```

3. **Install Python dependencies**:

```
pip install -r requirements.txt
```

4. **Set up PostgreSQL**:

```
# Install PostgreSQL and pgvector extension
# See POSTGRES_SETUP.md for detailed instructions

# Create database
createdb village_bank

# Enable pgvector extension
psql village_bank -c "CREATE EXTENSION IF NOT EXISTS vector;"
```

5. **Configure environment**:

```
# Copy example env file (if exists) or create .env in app/ directory
cd app
# Edit .env with your settings:
# - DATABASE_URL
# - JWT_SECRET_KEY
# - GROQ_API_KEY
# - OPENAI_API_KEY
# - SMTP settings (if using email)
```

6. **Run database migrations**:

```
cd ..
alembic upgrade head
```

7. **Seed initial data**:

```
python scripts/seed_data.py
python scripts/setup_ledger_accounts.py
```

8. **Start the backend server**:

```
cd app
source venv/bin/activate
```

```
uvicorn main:app --reload --port 8002
```

The API will be available at `http://localhost:8002` API documentation at `http://localhost:8002/docs`

Frontend Setup

1. **Navigate to UI directory**:

```
cd ui
```

2. **Install dependencies**:

```
npm install
```

3. **Configure API endpoint** (if needed): Edit `ui/lib/api.ts` to set the correct backend URL (default: `http://localhost:8002`)

4. **Start the development server**:

```
npm run dev
```

The UI will be available at `http://localhost:3000`

Key Workflows

1. Member Registration and Activation

```
Person registers on the app
  → Account created as "Pending"
  → Chairman reviews and approves
  → Member becomes "Active" and can participate
```

2. Monthly Dealing Dates (Activity Schedule)

Each month follows a fixed schedule of activity windows. Members receive an email notification when each window opens.

Activity	Opens	Closes	Description
Declaration Period	15th of the month	5th of the next month	Members submit monthly declarations (savings, social fund, admin fund, penalties, interest, loan repayment)

Activity	Opens	Closes	Description
Loan Application Period	21st of the month	25th of the month	Members can apply for new loans. Loan applications remain open throughout the month.
Deposit & Loan Repayment Period	25th of the month	5th of the next month	Members make payments to the bank account and upload proof of deposit

Example for March 2026:

- Declaration: 15 March – 5 April
- Loan Application: 21 March – 25 March
- Deposit & Repayment: 25 March – 5 April

Important notes:

- Declarations can only be created or edited within the declaration window (15th to 5th)
- Loan applications and repayments are open throughout the month
- Members are notified by email at 07:00 on each opening day
- The system enforces these windows — declarations outside the window are rejected

3. Monthly Declaration and Deposit Flow

Each month, members declare how much they are contributing and then prove they deposited it.

```
Member creates a Declaration for the month
  (Savings, Social Fund, Admin Fund, Penalties, Loan Repayment, Interest)
  → Status: Pending

Member uploads Deposit Proof (bank slip / screenshot)
  → Treasurer reviews the proof
  → If valid: Treasurer approves → funds posted to member's account
  → If invalid: Treasurer rejects with comment → member can resubmit
```

Key rules:

- One declaration per member per month
- Declarations can be edited within the declaration window (15th to 5th of next month)
- Savings balance only reflects approved deposits, not pending declarations
- The Chairman/Treasurer can also enter declarations via Reconciliation for backdating

4. Loan Flow

There are two ways a loan gets created:

Path A — Member applies (normal flow):

```
Member submits a Loan Application
  (amount, term, cycle)
  → System checks: credit rating, savings × multiplier = max allowed
  → Status: Pending

Treasurer reviews the application
  → Approves: Loan is created and money disbursed immediately
  → Rejects: Application marked rejected with reason

Member can also:
  → Edit a pending application (change amount/term)
  → Withdraw a pending application
```

Path B — Treasurer reconciliation (backlog/corrections):

```
Treasurer enters loan details directly via Reconciliation
  → Loan created immediately (no application needed)
  → Disbursement date set to the reconciliation month
```

Repayment:

```
Member declares loan_repayment (principal) and interest_on_loan each month
  → These are tracked through approved declarations
  → Outstanding balance = Loan Amount - Total Principal Paid

When fully paid (principal + interest both covered):
  → Loan automatically marked as "Paid Off"
  → Treasurer notified by email
```

Loan auto-closure happens in three ways:

- A background job runs every 5 minutes checking all active loans
- When the Treasurer views the active loans list
- When the Member views their current loan

5. Payment Requests & Expenses

The system supports a 3-step approval workflow for all payments and disbursements:

```
Step 1 — Initiation (Vice-Chairman or Chairman):
  → Creates a payment request with:
     - Amount
     - Description
     - Category (Committee Payment, Social Support, Admin Cost, End-of-
Year Payout)
     - Beneficiary (person, entity, or member for payouts)
```

```
    → Source account is automatically determined by category

  Step 2 — Approval (Chairman):
    → Reviews the request
    → Approves → proceeds to Treasurer
    → OR Rejects → returns with reason

  Step 3 — Execution (Treasurer):
    → Verifies sufficient balance in source account
    → Marks payment as executed
    → Journal entry posted (debit source fund, credit bank cash)
    → Records payment reference (bank ref, receipt number)
```

Payment Categories and Source Accounts:

Category	Source Account	Use Case
Committee Payment	Admin Fund	Honoraria, committee expenses
Social Support	Social Fund	Funerals, member emergencies
Administrative Cost	Admin Fund	Hosting, subscriptions, operational costs
End-of-Year Payout	Member Savings	Annual savings + interest distribution

Role Responsibilities:

Role	Can Do	Cannot Do
Vice-Chairman	Create requests, cancel own pending requests	Approve, reject, or execute
Chairman	Create requests, approve or reject pending requests	Execute payments
Treasurer	View approved requests, execute payments (post journal entry, record bank ref)	Create, approve, or reject

All three roles access Payment Requests from the main welcome dashboard.

Key rules:

- Source accounts cannot be overdrawn — balance is validated before execution
- Each category dropdown shows the current balance of the source fund
- End-of-year payouts debit the specific member's savings account
- Full audit trail: who initiated, approved, and executed each payment
- Only the initiator can cancel a pending request

Reports: The Payment Requests page includes a Reports tab with:

- Monthly summary cards (total requests, total amount, executed amount, pending amount)
- Breakdown by category (count and total per category)
- Breakdown by status (count and total per status)

- Full transaction list with filters by category and status, showing date, description, beneficiary, amount, status, and audit trail (initiated by, approved by, executed by, payment reference)
- Print support for physical records

6. Cycle Management

A cycle represents one financial year for the group.

```
Chairman creates a Cycle
  (year, start date, phases, credit rating scheme)
  → Phases define when declarations, loan applications, and deposits are
allowed
  → Each phase has date ranges and optional automatic penalties

Chairman activates the Cycle
  → Only one cycle can be active at a time
  → All member activity happens within the active cycle

Chairman assigns Credit Ratings to members
  → Each rating tier has a borrowing multiplier and interest rates
  → Example: "Low Risk A+" with 4x multiplier means max loan = 4 × savings

Chairman can close the Cycle when the year ends
```

7. Credit Rating and Loan Eligibility

```
Chairman assigns a Credit Rating tier to each member for the cycle
  → Tier determines:
     - Borrowing multiplier (e.g. 2x, 3x, 4x savings)
     - Available loan terms (1, 2, 3, or 4 months)
     - Interest rate per term

When a member applies for a loan:
  Maximum Loan = Savings Balance × Multiplier
  Interest = Loan Amount × Rate (flat, not compounding)
```

8. Penalty Management

```
Penalties can be applied:
  → Automatically: when a member acts outside allowed phase dates
    (late declaration, late deposit, late loan application)
  → Manually: by the Compliance Officer for other violations

Penalties go through:
  Created → Approved by Treasurer → Paid (via declaration)
```

9. Reconciliation (Backdating)

The Chairman or Treasurer can enter historical data for any past month.

```
Select a member + month → Load existing data (if any)
  → Enter: Savings, Social Fund, Admin Fund, Penalties,
           Loan Repayment, Interest, Loan Amount
  → Save: Declaration created, deposit approved, and posted to ledger
  → All entries are backdated to the selected month

Declarations can also be moved to a different month if entered
incorrectly.
```

10. AI Assistant

```
Members can chat with the AI assistant to:
  → Ask about their account (savings, loans, penalties)
  → Ask about constitution rules and policies
  → Get help navigating the app

Chairman and Treasurer get additional access:
  → Look up any member's financial details (savings, loans, penalties)
  → Look up member personal details (NRC, bank info, address)
```

11. Automated Background Tasks

The system runs automatic tasks on two schedules:

Every 5 minutes:

- **Auto-close paid loans** — Loans where principal and interest are fully paid get marked as "Paid Off"
- **Transfer excess contributions** — If a member overpays Social Fund or Admin Fund beyond the cycle requirement, the excess is transferred to their Savings
- **Treasurer notification** — Treasurers receive an email listing any loans closed or funds transferred

Daily at 07:00 (activity window notifications):

- **15th of each month** — All active members emailed that the Declaration Period is open
- **21st of each month** — All active members emailed that the Loan Application Period is open
- **25th of each month** — All active members emailed that the Deposit & Loan Repayment Period is open

---

Detailed Documentation

Chart of Accounts and System Rules

- See `docs/accounting_and_rules.md` for the complete account structure and business rules

API Endpoints

Authentication

- `POST /api/auth/register` - Register new user (requires admin approval)
- `POST /api/auth/login` - Login and get JWT token
- `GET /api/auth/me` - Get current user info

Member

- `GET /api/member/status` - Get account status (savings, loans, funds, penalties)
- `GET /api/member/cycles` - Get active cycles
- `POST /api/member/declarations` - Create monthly declaration
- `GET /api/member/declarations` - List all declarations
- `GET /api/member/declarations/current-month` - Get current month declaration
- `PUT /api/member/declarations/{id}` - Update declaration
- `POST /api/member/deposits/proof` - Upload deposit proof
- `GET /api/member/deposits` - List deposit proofs
- `POST /api/member/deposits/{id}/respond` - Respond to treasurer comment
- `GET /api/member/loans/eligibility/{cycle_id}` - Get loan eligibility
- `POST /api/member/loans/apply` - Apply for loan
- `GET /api/member/loans` - Get all loans (history)
- `GET /api/member/loans/current` - Get current active loan
- `PUT /api/member/loans/{id}` - Edit pending loan application
- `POST /api/member/loans/{id}/withdraw` - Withdraw loan application

Treasurer

- `GET /api/treasurer/deposits/pending` - List pending deposit proofs
- `POST /api/treasurer/deposits/{id}/approve` - Approve deposit and post to ledger
- `POST /api/treasurer/deposits/{id}/reject` - Reject deposit with comment
- `GET /api/treasurer/deposits/proof/{filename}` - View deposit proof file
- `GET /api/treasurer/penalties/pending` - List pending penalties
- `POST /api/treasurer/penalties/{id}/approve` - Approve penalty
- `GET /api/treasurer/loans/pending` - List pending loan applications
- `POST /api/treasurer/loans/{id}/approve` - Approve and disburse loan
- `GET /api/treasurer/loans/active` - List active loans
- `GET /api/treasurer/loans/{id}/details` - Get loan performance details

Chairman

- `GET /api/chairman/pending-members` - List pending members
- `GET /api/chairman/members` - List all members (with filters)
- `POST /api/chairman/members/{id}/approve` - Approve member
- `POST /api/chairman/members/{id}/suspend` - Suspend member
- `POST /api/chairman/members/{id}/activate` - Activate member
- `GET /api/chairman/cycles` - List all cycles
- `POST /api/chairman/cycles` - Create new cycle

- `GET /api/chairman/cycles/{id}` - Get cycle details
- `PUT /api/chairman/cycles/{id}` - Update cycle
- `POST /api/chairman/cycles/{id}/activate` - Activate cycle
- `POST /api/chairman/cycles/{id}/close` - Close cycle
- `POST /api/chairman/cycles/{id}/reopen` - Reopen closed cycle
- `GET /api/chairman/users` - List all users
- `POST /api/chairman/members/{id}/credit-rating` - Assign credit rating
- `GET /api/chairman/credit-rating-tiers/{cycle_id}` - Get credit rating tiers

Admin

- `GET /api/admin/settings` - Get system settings
- `PUT /api/admin/settings` - Update system settings

Payment Requests

- `POST /api/payment-requests/` - Create payment request (Vice-Chairman, Chairman)
- `GET /api/payment-requests/` - List payment requests with optional status filter
- `GET /api/payment-requests/{id}` - Get payment request detail
- `PUT /api/payment-requests/{id}/approve` - Chairman approves
- `PUT /api/payment-requests/{id}/reject` - Chairman rejects with reason
- `PUT /api/payment-requests/{id}/execute` - Treasurer executes and posts journal entry
- `PUT /api/payment-requests/{id}/cancel` - Cancel pending request (initiator only)
- `GET /api/payment-requests/account-balances` - Get fund balances (Admin Fund, Social Fund, Bank Cash)
- `GET /api/payment-requests/reports/summary` - Monthly summary (totals by status and category)
- `GET /api/payment-requests/reports/transactions` - Detailed transaction list with audit trail

AI Chat

- `POST /api/ai/chat` - AI chat endpoint (RAG with constitution/policies)

Database Schema

Core Tables

- `user` - User accounts (preserved from legacy system)
- `member_profile` - Member profiles linked to users
- `user_role` - Role assignments
- `cycle` - Financial cycles
- `cycle_phase` - Cycle phases

Accounting Tables

- `ledger_account` - Chart of accounts
- `journal_entry` - Journal entry headers
- `journal_line` - Journal entry lines (double-entry)

Transaction Tables

- `declaration` - Member monthly declarations
- `deposit_proof` - Deposit proof documents
- `deposit_approval` - Treasurer approvals
- `loan_application` - Loan applications
- `loan` - Approved loans
- `repayment` - Loan repayments
- `penalty_record` - Penalty records

Policy Tables

- `credit_rating_scheme` - Credit rating schemes
- `credit_rating_tier` - Rating tiers
- `member_credit_rating` - Member ratings per cycle
- `credit_rating_interest_range` - Interest rates per tier
- `borrowing_limit_policy` - Borrowing limits

Migration from Legacy System

See `docs/migration/` for detailed migration guides:

- `mapping.md` - Field mappings
- `rules.md` - Business rules
- `validation_checklist.md` - Validation steps

Migration scripts:

- `scripts/export_old_schema.sh` - Export old schema
- `scripts/migrate_load.py` - Load staging data
- `scripts/migrate_transform.py` - Transform to journals
- `scripts/migrate_validate.py` - Validate migration

Development

Running Tests

```
# Backend tests (when implemented)
pytest

# Frontend tests (when implemented)
cd ui
npm test
```

Creating Migrations

```
alembic revision --autogenerate -m "Description of changes"
alembic upgrade head
```

Code Style

- Backend: Follow PEP 8, use type hints
- Frontend: ESLint configuration included

Deployment

Production Considerations

- Set `ENVIRONMENT=production` in `.env`
- Use strong `JWT_SECRET_KEY`
- Configure proper CORS origins
- Set up SSL/TLS
- Use production database with backups
- Configure proper file storage for uploads
- Set up monitoring and logging

License

Proprietary - LUBOSS 95

Support

For issues and questions, refer to:

- `TROUBLESHOOTING.md` - Common issues and solutions
- `QUICK_START.md` - Quick setup guide
- `SETUP_MAC.md` - Mac-specific setup
- `POSTGRES_SETUP.md` - PostgreSQL setup guide

Version History

v1.0.0 (Stable Release)

- Complete loan management system
- Member declarations with deposit proof workflow
- Treasurer dashboard with loan approval and monitoring
- Chairman dashboard with cycle and user management
- Credit rating system with borrowing limits
- Full accounting ledger integration
- AI chat with RAG (constitution/policies)
- Mobile-first responsive UI